Aeronautics and Space Technology Club

IIT (ISM) Dhanbad

Signal Processing Ladder: A Hands-On Problem Series from Sensors to Machine Learning

Problem Statement 20 · Part D: Digital Signal Processing Level 4

Track: LMS, NLMS, RLS, Noise Cancellation, Kalman Filter, IMU and GPS Sensor Fusion

Prerequisites: Problems 06, 07, 14 · **Estimated effort:** 16–22 hours

Adaptive Filters and Kalman Estimation: Filters That Learn, and Estimators That Know Their Own Uncertainty

When the noise statistics are not known in advance, or the quantity of interest is a hidden state, fixed filters are not enough. Implement LMS, NLMS and RLS, cancel engine noise from a sensor, write a Kalman filter from scratch, fuse a drifting accelerometer with GPS, ride through a GPS outage, and check that your filter's uncertainty is honest.

General Instructions

- **Thorough Review:** Read the whole problem statement, including the theory, before writing any code. Do the short analytic derivations *by hand first*; the code is there to confirm and extend your understanding, not to replace it.
- **Pacing:** This is a self-paced problem. The split below is a guide for the effort estimate in the header.

Stage	Phase	What you should have at the end of it
1	Think (∼20%)	Theory understood; every analytic result derived; signals, units and sampling rates written down.
2	Build (∼30%)	Core code working end to end on the baseline data and validated against the analytic results.
3	Experiment (∼30%)	Full sweeps, stress tests and the real or synthetic datasets processed; first plots and numbers.
4	Refine (∼20%)	Weaknesses fixed, honest analysis and memo written, repository folder clean.

- **Compute and Tools:** Everything is designed to run on a laptop CPU in Python 3.10+ (NumPy, SciPy, Matplotlib, and where stated pandas, Astropy, scikit-learn, PyWavelets and PyTorch). No GPU and no hardware are required; where a problem offers an optional hardware experiment it is marked as a bonus. A full run should never need more than a few minutes; if yours does, vectorise before you scale up.
- **Data:** Each problem lists (i) a *synthetic generator* from `sp_common.synth` (offline, seeded, with known ground truth) and (ii) *real datasets* to download yourself. Develop and validate on the synthetic data first, then repeat the key result on real data. Never commit large raw data files to the repository.
- **Submission Format:** Push your work to `solutions/20_adaptive_filters_and_kalman_estimation/` in the repository: (i) clean, commented, reproducible Jupyter notebook(s) with fixed random seeds, (ii) a short written report (`report.pdf` or `report.md`) containing the requested analysis

sections, and (iii) all figures in a `figures/` sub-folder. The report must be understandable without running the code.

- **Originality:** All code, analysis and written explanations must be your own work. Any external library, snippet, paper or AI tool you use must be cited in the report. You should be able to explain every line you submit.
- **Honesty over Hype:** Validate against analytic results *before* trusting a number. A modest or negative result that is correctly analysed is worth more than an inflated claim. Report error bars, false-alarm rates and failure cases, not only your best run.
- **Units:** Write the units of every quantity and state in the first notebook cell whether a frequency is in Hz or rad/s and whether it is normalised to the sampling rate. Most mistakes in signal processing are factors of 2π, factors of 2 in one-sided spectra, or an off-by-one in the frequency axis.
- **Getting Stuck:** Re-read the theory section, check the self-check targets, then ask your mentor or open an issue in the repository. Describe what you expected, what you got, and what you already tried.

Background and Motivation

A fixed filter embodies assumptions about the signal and noise. Adaptive filters adjust their own coefficients to the data: they track a changing system, cancel interference whose waveform is unknown but correlated with a reference, and identify unknown channels. The Kalman filter takes the idea further: given a model of how a hidden state (position, velocity, sensor bias) evolves and how it is measured, it produces the optimal estimate and, equally important, its *covariance*, a statement of how much to trust the answer.

Navigation is the textbook application. An inertial sensor is excellent over seconds and terrible over minutes (its error grows quadratically with time); GPS is noisy but drift-free and can be lost. Fusing them with a Kalman filter gives a solution better than either, and it keeps working, with growing uncertainty, through an outage. The noise parameters you need come from the sensor analysis of problems 01 and 07.

Theoretical Framework

1. Wiener Solution and LMS

For a tap-weight vector $\boldsymbol{w}$ filtering the regressor $\boldsymbol{x}[n] = (x[n], \dots, x[n-M+1])^\top$ to estimate $d[n]$, the mean-square error is minimised by the normal equations $\mathbf{R}\boldsymbol{w}_o = \boldsymbol{p}$ with $\mathbf{R} = \mathbb{E}\{\boldsymbol{x}\boldsymbol{x}^\top\}$, $\boldsymbol{p} = \mathbb{E}\{d\,\boldsymbol{x}\}$. The least-mean-squares algorithm replaces the expectations by instantaneous values:

$$e[n] = d[n] - \boldsymbol{w}^\top \boldsymbol{x}[n], \qquad \boldsymbol{w} \leftarrow \boldsymbol{w} + \mu\, e[n]\, \boldsymbol{x}[n], \qquad 0 < \mu < \frac{2}{\lambda_{\max}} \left(\lesssim \frac{2}{M\sigma_x^2} \right), \tag{1}$$

with misadjustment about $\mu M \sigma_x^2/2$ and convergence limited by the eigenvalue spread of $\mathbf{R}$. NLMS divides the step by $\epsilon + \|\boldsymbol{x}\|^2$. RLS minimises an exponentially weighted least-squares cost (forgetting factor λ_f) and converges much faster at a cost of $\mathcal{O}(M^2)$ per sample.

2. Adaptive Noise Cancellation

A primary sensor records $s + n_0$ (wanted signal plus noise); a reference sensor records n_1, correlated with n_0 through an unknown path but not with s. An adaptive FIR driven by n_1 learns that path; the error output $e = s + n_0 - \hat{n}_0$ converges to s.

3. Kalman Filter

For $\boldsymbol{x}_k = \mathbf{F}\boldsymbol{x}_{k-1} + \mathbf{B}\boldsymbol{u}_k + \boldsymbol{w}_k$ ($\boldsymbol{w} \sim \mathcal{N}(0, \mathbf{Q})$) and $\boldsymbol{z}_k = \mathbf{H}\boldsymbol{x}_k + \boldsymbol{v}_k$ ($\boldsymbol{v} \sim \mathcal{N}(0, \mathbf{R})$):

$$\begin{aligned} \hat{\boldsymbol{x}}_{k|k-1} &= \mathbf{F}\hat{\boldsymbol{x}}_{k-1} + \mathbf{B}\boldsymbol{u}_k, & \mathbf{P}_{k|k-1} &= \mathbf{F}\mathbf{P}_{k-1}\mathbf{F}^\top + \mathbf{Q}, \\ \mathbf{K}_k &= \mathbf{P}_{k|k-1}\mathbf{H}^\top(\mathbf{H}\mathbf{P}_{k|k-1}\mathbf{H}^\top + \mathbf{R})^{-1}, & \hat{\boldsymbol{x}}_k &= \hat{\boldsymbol{x}}_{k|k-1} + \mathbf{K}_k(\boldsymbol{z}_k - \mathbf{H}\hat{\boldsymbol{x}}_{k|k-1}), \quad \mathbf{P}_k = (\mathbf{I} - \mathbf{K}_k\mathbf{H})\mathbf{P}_{k|k-1}. \end{aligned} \tag{2}$$

For a constant-velocity model with sample time Δt, $\mathbf{F} = \left(\begin{smallmatrix} 1 & \Delta t \\ 0 & 1 \end{smallmatrix}\right)$ and white-acceleration noise of spectral density q, $\mathbf{Q} = q\left(\begin{smallmatrix} \Delta t^3/3 & \Delta t^2/2 \\ \Delta t^2/2 & \Delta t \end{smallmatrix}\right)$. With a measured acceleration a_k as input, $\mathbf{B} = (\Delta t^2/2,\ \Delta t)^\top$. Appending a bias state b (random walk) lets the filter estimate and remove an accelerometer bias.

4. Consistency

The *innovations* $\boldsymbol{\nu}_k = \boldsymbol{z}_k - \mathbf{H}\hat{\boldsymbol{x}}_{k|k-1}$ of a correct filter are zero-mean, white, with covariance $\mathbf{S}_k = \mathbf{H}\mathbf{P}_{k|k-1}\mathbf{H}^\top + \mathbf{R}$. The normalised innovation squared $\mathrm{NIS}_k = \boldsymbol{\nu}_k^\top \mathbf{S}_k^{-1}\boldsymbol{\nu}_k$ is χ^2 distributed with $\dim \boldsymbol{z}$ degrees of freedom; with ground truth the normalised estimation error squared $\mathrm{NEES}_k = (\boldsymbol{x}_k - \hat{\boldsymbol{x}}_k)^\top \mathbf{P}_k^{-1}(\boldsymbol{x}_k - \hat{\boldsymbol{x}}_k)$ is χ^2 with $\dim \boldsymbol{x}$ degrees of freedom.

5. Dead-Reckoning Drift

Integrating an accelerometer with constant bias b twice gives a position error $\frac{1}{2}bt^2$; integrating white noise of density σ_a gives a velocity random walk $\sigma_a\sqrt{t}$.

Data and Simulation Specification

System identification	Unknown FIR with 8 taps, white input of variance 1 and an AR(1) coloured input with $a = 0.9$; observation noise $\sigma = 0.1$; $M = 8$ and $M = 16$ taps.
Noise cancellation	A 12 Hz vibration signal of amplitude 1 plus engine noise (a 80 Hz harmonic series plus broadband noise) filtered by an unknown 16-tap path; the reference sensor sees the engine noise with its own small noise.
1-D tracking	Target with true velocity 5 m/s and random acceleration $q = 0.1$; position measurements every 0.1 s with $\sigma = 2$ m.
IMU and GPS	`sp_common.synth.imu_gps_trajectory(duration=120, fs_imu=100, fs_gps=1, accel_noise=0.15, accel_bias=0.08, gps_noise=3.0, gps_outage=(60,80))`: planar motion, accelerometer with bias, GPS with a 20-s outage, and the truth.
Real data (choose one)	EuRoC MAV dataset (IMU at 200 Hz, accurate ground-truth position): build "GPS-like" fixes at 1 Hz with 3 m noise from the ground truth and run your filter; or record ADS-B aircraft positions with an RTL-SDR and `dump1090` and smooth the tracks.

Your Task

1. **Wiener, LMS, NLMS, RLS (Stage 2).** Solve the normal equations for the identification problem and implement LMS, NLMS and RLS. Plot learning curves (mean-square error versus iteration, averaged over 200 runs) for several μ, verify the stability bound of Eq. (1), measure the misadjustment against $\mu M \sigma_x^2/2$, and show how colouring the input (eigenvalue spread of $\mathbf{R}$) slows LMS but barely affects RLS.

2. **Noise cancellation.** Build the canceller with NLMS. Measure the noise reduction in dB, the distortion of the 12 Hz signal and the convergence time; show what happens to the output when the reference sensor also picks up some of the wanted signal.
3. **Kalman filter in one dimension.** Implement Eq. (2) for the constant-velocity tracker. Compare position and velocity errors with a moving average and a finite difference. Iterate the Riccati equation to the steady-state gain and compare with the α–β filter. Test innovation whiteness (autocorrelation within $\pm 1.96/\sqrt{N}$) and NIS bounds.
4. **IMU and GPS fusion.** Implement the planar filter with state (x, y, v_x, v_y), the accelerometer as control input at 100 Hz and GPS updates at 1 Hz. Derive $\mathbf{Q}$ from σ_a (white-noise acceleration) and $\mathbf{R}$ from the GPS noise. Compare the position RMSE of the filter, of GPS alone (interpolated) and of pure dead reckoning.
5. **Bias estimation and the outage.** Add accelerometer-bias states with random-walk noise. Show the bias estimate converging, the position error through the 60 s–80 s outage with and without bias states, and plot the covariance with 3σ envelopes. Determine how long an outage the filter can bridge while keeping the position error below 10 m.
6. **Tuning and consistency.** Sweep $\mathbf{Q}$ by factors 10^{-2} to 10^{2}; compute average NEES and NIS with ground truth over 50 runs and identify over- and under-confident filters. Relate $\mathbf{Q}$ to the Allan-variance parameters of problem 01 (white noise to σ_a, random walk to the bias process noise).
7. **Real data.** Run the fusion filter on EuRoC (or ADS-B) data, report the position RMSE against ground truth and the NIS statistics, and discuss which model assumptions fail on real motion.

Self-check targets

Pure dead reckoning with a constant bias $b = 0.08\,\mathrm{m/s^2}$ drifts as $\frac{1}{2}bt^2$: 144 m after 60 s, 576 m after 120 s. The LMS step must satisfy $\mu < 2/\lambda_{\max}$; for unit-variance white input and $M = 8$, $\mu < 0.25$ is the practical limit. A consistent filter has time-averaged NIS equal to $\dim \boldsymbol{z}$ (2 for planar GPS) and NEES equal to $\dim \boldsymbol{x}$. A converged filter's innovations are white to within the $\pm 1.96/\sqrt{N}$ band.

Aerospace Application Link

A small UAV must keep its position error under 10 m for a 60 s GPS outage. Dead reckoning from an accelerometer with constant bias b needs $\frac{1}{2}bt^2 < 10$, i.e. $b < 2 \cdot 10/60^2 = 5.6 \times 10^{-3}\,\mathrm{m/s^2}$ (0.57 mg) *without* bias estimation. Using your Task 5 results state which accelerometer grade is required with and without bias states, how long the filter can bridge outages in each case, and what the covariance tells the flight computer about when to stop trusting the navigation solution.

Deliverables

D1 **Adaptive filters:** Wiener solution, LMS/NLMS/RLS learning curves, stability bound and misadjustment checks.

D2 **Noise cancellation:** the canceller with reduction in dB, signal distortion and the failure case.

D3 **Kalman 1-D:** implementation, steady-state gain, innovation and NIS tests.

D4 **IMU/GPS fusion:** the planar filter, RMSE comparison and the outage bridging analysis with covariance envelopes.

D5 **Tuning, consistency and real data:** NEES/NIS sweep over **Q**, the link to Allan variance, EuRoC or ADS-B result.

D6 **Memo (300 words):** "What IMU grade and filter do we need for a 60-second GPS outage?" Cite at least two external sources.

Bonus (a) Implement an extended Kalman filter for range–bearing measurements. (b) Implement a Rauch–Tung–Striebel smoother and quantify the gain over the filter. (c) Replace the Kalman filter by a particle filter for a non-Gaussian measurement noise and compare.

Learning Resources

Adaptive Filtering

- Haykin, S. *Adaptive Filter Theory.*
- Widrow, B., Stearns, S.D. *Adaptive Signal Processing* (noise cancelling).
- Widrow, B. et al. (1975). "Adaptive noise cancelling: principles and applications." Proc. IEEE 63, 1692.

Kalman Filtering and Navigation

- Kalman, R.E. (1960). "A new approach to linear filtering and prediction problems." J. Basic Eng. 82, 35.
- Labbe, R. *Kalman and Bayesian Filters in Python* (free online book on GitHub, with notebooks).
- Bar-Shalom, Y., Li, X.R., Kirubarajan, T. *Estimation with Applications to Tracking and Navigation* (NIS, NEES, tuning).
- Groves, P.D. *Principles of GNSS, Inertial, and Multisensor Integrated Navigation Systems.*

Data and Tools

- EuRoC MAV dataset: `https://projects.asl.ethz.ch/datasets/doku.php?id=kmavvisualinertialdata` OpenSky Network: `https://opensky-network.org`
- NumPy linear algebra and SciPy: `https://numpy.org/doc/stable/reference/routines.linalg.html`

Evaluation

Score your own submission out of 100 on five equally weighted criteria, then ask your mentor to score it independently and compare.

Criterion	What to look for	Marks
Conceptual depth	Correct mathematics and physics; every result explained through a mechanism (why the spectrum looks like this), not only through code behaviour.	/20
Implementation quality	Correct, modular, commented, reproducible code; no unexplained magic numbers; analytic checks and self-check targets pass.	/20
Aerospace-application integration	The signal-processing result is carried through to a measurable consequence for a sensor, a link, a flight system or a detection decision, as the task requires.	/20
Analysis and critical thinking	Error bars, convergence, comparison to theory, honest treatment of limitations, false alarms and failure cases.	/20
Communication	Clear report, labelled plots (units, legends, captions) and a memo that a non-expert student could follow.	/20

Bonus tasks are not part of the 100 marks. A bonus attempted badly does not cost you marks, and a well-executed one is the best sign that you own the material.